

Structural Paradigms in Autonomous Scripting: A Comparative Analysis of JonWinsley's Grey Company, Overmind, and Competitive Screeps Architectures

The development of autonomous agents within the Screeps ecosystem represents a unique frontier in computational strategy and distributed systems design. As a persistent, real-time strategy environment where the core mechanic is the deployment of JavaScript or TypeScript code, Screeps demands a level of architectural rigor typically reserved for industrial software applications.¹ Within this domain, the evolution of player-built AI has branched into several distinct philosophies, ranging from the deeply hierarchical, object-oriented approach of the Overmind framework to the performance-centric, objective-driven model pioneered by JonWinsley, also known as glitchassassin.¹ These architectures are not merely sets of instructions but comprehensive management systems designed to handle resource logistics, defensive posturing, and multi-room coordination under the strict constraints of server-side CPU limits and memory parsing costs.⁴ Understanding the trade-offs between these systems requires an exploration of their underlying design principles, their handling of game-state persistence, and their adaptability to the shifting meta-game of the public server environment.

The Overmind Framework: Hierarchical Complexity and Collective Intelligence

The Overmind AI stands as perhaps the most influential open-source project in the Screeps community, characterized by its thematic cohesion and ambitious structural abstractions.¹ Developed by Ben Bartlett (Muon), Overmind is composed of approximately 40,000 lines of TypeScript, organized around a Zerg-inspired biological metaphor where various tiers of responsibility dictate creep behavior.¹ This hierarchical approach is designed to abstract the immense complexity of controlling a large-scale empire, allowing a player to manage thousands of units through high-level directives rather than low-level creep logic.⁷

The Hierarchy of Responsibility

The architectural spine of Overmind is a tree of specialized objects, each responsible for a specific scope of the colony's operations. At the apex is the Overmind singleton, which wraps all game objects and manages the initialization of the colony hierarchy.⁷ Beneath the global controller, the system is partitioned into Colonies, representing an owned room and its associated outposts.⁷ Within each colony, the architecture further decomposes into

HiveClusters and Overlords.⁷

HiveClusters serve as the organizational units for structures that share a common purpose, such as the Hatchery for creep production or the Command Center for resource management.⁷ Overlords, meanwhile, act as the functional "brains" for specific tasks, managing the spawning and behavior of creeps (wrapped in a specialized "Zerg" class) to achieve localized goals.⁷ This structure allows for a high degree of modularity; an Overlord can be attached to a Colony for long-term objectives or to a Directive—a wrapper for a game flag—for temporary operations like defending against an invasion or claiming a new territory.⁷

Computational Evolution and the Refresh Paradigm

One of the primary historical drawbacks of the Overmind architecture was its significant CPU overhead. The deep object-oriented structure, which relies heavily on classes, originally required the recursive instantiation of the entire hierarchy during every game tick.⁸ In the Node.js environment that powers Screeps, this constant instantiation led to excessive processing time and high garbage collection costs, particularly as the number of rooms and creeps grew.⁸

To address this, the architecture transitioned to a "refreshable" model. The modern Overmind execution loop is divided into three primary phases: build(), init(), and run().⁸ The heavy build() phase, which instantiates the hierarchy, is now run only once every 20 ticks by default.⁸ On all other ticks, the system executes a refresh() phase, which updates references to game objects in-place without re-instantiating the script objects.⁸ This "soft update" approach, facilitated by a specialized caching module (the \$ module), reportedly reduced CPU consumption by over 40% while maintaining the benefits of a structured, object-oriented design.⁸

Phase	Frequency	Core Responsibility
build()	Every n ticks ($n = \quad$)	Recursive instantiation of Colonies, HiveClusters, and Overlords. ⁸
refresh()	Every tick ($n \neq \quad$)	In-place update of game object references and property caching. ⁸
init()	Every tick	Registration of intent for spawning, resource logistics, and repairs. ⁷

run()	Every tick	Execution of state-changing actions, creep direction, and intel gathering. ⁸
-------	------------	---

The Social and Strategic Meta of Assimilation

Overmind is unique in its implementation of the "Assimilation" protocol, a feature that enables multiple players running the codebase to coordinate as a single collective hivemind.¹ Through a decentralized ledger and a verification system involving sha256 cryptographic hashes to ensure codebase compatibility, "assimilated" players can share resource requests and military directives.⁸ This capability introduced a novel social dynamic to the game, as coordinated swarms of Overmind bots could dominate large regions of the map.⁶ However, this popularity has also made the architecture a target; because the code is open-source and its behaviors are well-documented, "Overmind hunting" groups have emerged, utilizing the bot's predictable heuristics and remote mining vulnerabilities to systematically dismantle its empires.⁹

JonWinsley's Grey Company: Performance-Oriented Lean Architectures

Jon Winsley (glitchassassin) represents an architectural shift toward extreme efficiency and the reduction of unnecessary abstractions.³ His primary codebase, the Grey Company, was born out of a pivot away from complex hierarchical classes like "Boardroom" and "Office" after discovering that such systems often introduced obscure dependencies and performance bottlenecks.³ This philosophy, described as the "Discrete Objectives" model, prioritizes simplicity, explicit state management, and the minimization of game intents.³

Discrete Objectives vs. Hierarchical Inheritance

The core of the Grey Company architecture is the Objective, which serves as a unified controller for task management, spawning, and unit behavior.³ Instead of separating these concerns into different layers of a hierarchy, an Objective is responsible for monitoring its own requirements, spawning the necessary units, and directing them until the goal is achieved.³ This flatter structure eliminates the CPU cost associated with navigating deep class trees and reduces the complexity of the main game loop to three primary iterations: over visible rooms, over cached Offices, and over all creeps.³

This lean approach has yielded significant performance gains. JonWinsley reported reducing CPU usage from approximately 11 CPU per room to just 4 CPU, a critical advantage on CPU-restricted shards like shard3 where efficiency is the primary constraint.³ By avoiding the "overengineering" typical of high-level frameworks, the Grey Company allows for faster startup

times and more scalable multi-room expansion.³

Technological Integration: Rust, WASM, and Advanced Pathfinding

A defining characteristic of JonWinsley’s technical strategy is the integration of high-performance modules outside the standard JavaScript environment. Most notable is the development of a custom pathfinding library written in Rust and compiled to WebAssembly (WASM).³ This library was designed to outperform the default Screeps pathfinder, which, although optimized by the developers, remains a significant source of CPU consumption for large empires.⁵

Efficient pathfinding is central to the Grey Company’s economic strategy, particularly on shard3 where movement is minimized to save energy and CPU.⁵ The bot utilizes a modular room planning system that distinguishes between "Franchises" and "Headquarters".⁵ Franchises are specialized harvesting outposts placed at energy sources, consisting of a spawn, a link, and a container to maximize throughput while minimizing the distance creeps must travel.⁵

Specialized Unit Roles and Logistics

The Grey Company defines a set of highly specialized roles, each with a narrow scope of operation to minimize wasted CPU on decision-making.¹¹ These roles are categorized into core economic functions and specialized military groups:

- **Remote Miner and Runner:** A decoupled system where the miner stays at the source and the runner transports energy, ensuring that move intents are only used by the hauler.¹¹
- **Transporter:** A dedicated role for redistributing energy within the base to structures such as extensions, towers, and factories.¹¹
- **Ferry:** A specialized operator for advanced tasks including lab management and terminal operations.¹¹
- **Quads:** A military unit consisting of four creeps that move as a single entity, utilizing coordinated healing and focused attacks to dismantle enemy defenses.¹¹

Role Category	Primary Purpose	Key Architectural Implication
Economic Core	Sustainable energy harvesting and distribution.	Minimizes move intents through static mining and links. ⁵
Infrastructure	Construction, maintenance, and RCL progression.	Scales with available energy surplus via the energyValue

		metric. ³
Military/Combat	Defense, harassment, and structural destruction.	Uses specialized flags like quadRally and break for coordinated strikes. ¹¹

The BonzAI Framework: Framework-Driven Automation and Evolutionary Design

The BonzAI codebase, developed by bonzaiferroni, provides an alternative framework-driven approach that balances structure with flexibility.¹³ Explicitly designed for the MMO server, BonzAI separates the AI into abstract framework classes and concrete implementations, allowing for significant code reuse and consistent execution phases.¹³

The Operation and Mission Paradigm

BonzAI is built around the concepts of Operations and Missions.¹³ An Operation represents a high-level strategic goal, such as claiming a room or defending a sector, while Missions are the building blocks that define the specific behaviors and spawning conditions required to execute that operation.¹³ This design ensures that every task has a consistent lifecycle, moving through initialization, role call, and execution phases in a predictable order.¹³

This structure is particularly effective for managing the diverse set of tasks required at higher Room Controller Levels (RCL). For instance, an operation to manage a Source Keeper (SK) room might utilize a BodyguardMission for defense, a MiningMission for energy harvesting, and a PaverMission for road maintenance.¹³ By encapsulating these behaviors into discrete mission classes, the framework allows the AI to react to complex environmental stimuli without requiring a specialized script for every possible scenario.¹³

Evolutionary Goals and Autonomous Expansion

A core aspiration of the BonzAI project is the implementation of evolutionary decision-making.¹³ The developer aims to create a system where the AI's behavior changes based on the results of previous decisions or through random mutations.¹³ Currently, this is evidenced in systems like AutoOperation, which can analyze nearby sources and room layouts to autonomously determine the optimal placement for base structures.¹⁴ This move toward full autonomy distinguishes BonzAI from more manual, flag-driven architectures, as it attempts to remove the human element from expansion and economic management.¹⁴

Screeps-Quorum: Community Governance and Decentralized Intelligence

Screeps-Quorum occupies a unique position as a community-driven, fully automated bot governed by a decentralized consensus model.¹⁶ Unlike other architectures maintained by single developers, Quorum is a collaborative effort where all pull requests are voted on by the community and automatically deployed once merged.¹⁶

Transparency and Non-Partisan Behavior

Quorum is designed to function as an "NPC" within the Screeps world—a neutral entity that anyone can test their code against.¹⁵ To maintain this neutrality, the project excludes all alliance-related code, ensuring that the bot is as likely to attack an ally as any other player.¹⁵ Furthermore, Quorum provides unique controller signatures and creep names, making it easily identifiable for bot-detection scripts.¹⁵ This transparency is a core part of its mission to act as a benchmark for the community rather than a vehicle for a single player's dominance.¹⁵

Technical Resilience through Consensus

The technical challenges of a community-run bot are significant, as contributions come from developers with varying levels of expertise. To manage this, Quorum utilizes rigorous testing and automated deployment pipelines.¹⁶ Its architectural discussions often center on efficiency-focused designs, such as "extension flowers" for CPU savings and the use of the `sos.lib.vram` module for persistent data storage in memory segments.¹⁷ This ensures that even with a decentralized contributor base, the bot remains competitive on the public server.¹⁶

Comparative Technical Analysis: CPU, Memory, and Intent Optimization

The effectiveness of these architectures is ultimately measured by their performance on the Screeps server, where CPU is a finite and expensive resource. Every game tick represents a 1ms window of processor time on the server, and every successful intent—such as moving, harvesting, or attacking—costs a flat 0.2 CPU.⁴

Caching Strategies and JSON Parsing Costs

A major bottleneck in Screeps performance is the cost of parsing the Memory object. Because persistent data is stored as a JSON string and parsed into a JavaScript object at the start of each tick, a large or deeply nested memory structure can consume several CPU units before the AI even begins to execute its logic.¹⁹

The different architectures handle this through varied caching mechanisms:

1. **Overmind** uses a global caching module to store game objects in the heap between ticks, only re-instantiating them during the `build()` phase every 20 ticks to minimize parsing and instantiation overhead.⁸

2. **Grey Company** favors a flatter memory structure and uses Names/Ids instead of game objects to reduce the volume of data that must be stringified and parsed.³
3. **Traveler** (a common movement library used by many bots, including BonzAI) caches paths in creep memory to avoid expensive PathFinder calls, though it introduces the trade-off of increased memory usage.²¹

Intent Management and Economic Efficiency

Efficient bots prioritize actions that provide the highest return for their CPU cost. In Screeps, "space is cheap and time is expensive," leading developers to favor strategies that use memory to save computation.²³ For example, a creep mining energy and transferring it to a link can save CPU by waiting until its store is full rather than transferring every tick, thereby reducing the number of 0.2 CPU intent calls.¹⁰

JonWinsley's architecture excels in this area by integrating these optimizations directly into its "Discrete Objectives".³ By calculating the energyValue of each unit per tick, the AI can scale its workforce to ensure that it only spawns enough units to maintain a surplus, avoiding the "CPU leak" caused by idle or inefficient creeps.³

Combat and Defense: Squad Logic and Tower Intelligence

The military meta in Screeps has shifted from simple defender scripts to complex, multi-creep maneuvers. This transition has forced architectures to adapt their internal logic to handle coordinated units and intelligent defensive responses.

Active Defensive Buffers and Min-Cut Perimeters

Defending a room requires more than just building walls; it requires an "active" defense that prevents attackers from draining resources. JonWinsley's Grey Company employs an implementation of the min-cut algorithm to generate optimal rampart placements that minimize the perimeter while ensuring structures are protected from ranged attacks.¹² This is paired with "Active Towers" that intelligently target attackers based on their potential healing.¹² If an enemy creep has enough healing from itself or nearby allies to tank the tower's fire, the tower will ignore it to preserve energy, forcing the attacker to commit more units rather than simply draining the defense.¹²

The Proliferation of Quads

In offensive combat, the "Quad" has become a standard tactical unit. Coordinating four creeps to move and act in unison requires a sophisticated state machine and high-performance pathing.⁵ Overmind manages this through specialized military directives, while Grey Company integrates quad logic directly into its combat objectives.⁷ The challenge for any architecture is

maintaining the formation across room transitions and in tight quarters, which demands a level of precision in creep movement that the default moveTo function often fails to provide.³

Architectural Pros and Cons Summary

The trade-offs between these architectures can be summarized by their adherence to different performance and organizational paradigms.

Architecture	Primary Pros	Primary Cons
Overmind	High-level automation; social collective features; visually polished with extensive room visuals. ¹	High hierarchical complexity; predictable and well-documented behaviors are easily exploitable. ⁹
Grey Company	Extreme CPU efficiency (4 CPU/room); Rust/WASM optimized pathfinding; lean state-machine logic. ³	Less accessible for novice players; requires significant understanding of low-level mechanics to modify. ³
BonzAI	Robust framework for code reuse; automated expansion via AutoOperation; educational structure. ¹³	Higher initial configuration time; some features like the Empire class are historically less mature. ¹³
Screeps-Quorum	Decentralized and resilient community development; transparent and neutral behavior. ¹⁵	Potential for fragmented strategy; performance depends on the quality of diverse community contributions. ¹⁷

Conclusion: The Trajectory of Persistent Agent Design

The architectural landscape of Screeps is a testament to the sophistication possible in automated systems design under constraint. Overmind has demonstrated that high-level abstractions and collective intelligence can create powerful, empire-scale agents that dominate the social and economic meta of the public servers.¹ However, the rise of performance-centric models like JonWinsley’s Grey Company highlights a counter-trend toward lean, optimized architectures that can operate across dozens of rooms with minimal

CPU overhead.³

As the game environment continues to mature—marked by the introduction of intershard travel, power creeps, and more complex server-side modifications—the most successful architectures will likely be those that can synthesize these two approaches.¹ The future of Screeps AI lies in the integration of high-level strategic management with low-level execution efficiency, perhaps utilizing specialized modules in Rust or WASM for critical calculations while maintaining a flexible, mission-based framework for high-level decision making.³ For the developer, the choice of architecture remains a fundamental decision that dictates not only their success in the game world but also their approach to solving the complex logistical and tactical problems inherent in a persistent, automated environment.

Works cited

1. bencbartlett/Overmind: AI for Screeps, a multiplayer programming strategy game - GitHub, accessed February 24, 2026, <https://github.com/bencbartlett/Overmind>
2. A standalone server for programming game Screeps - GitHub, accessed February 24, 2026, <https://github.com/screeps/screeps>
3. Screeps #20: The Great Purge - Field Journal, accessed February 24, 2026, <https://jonwinsley.com/notes/screeps-great-purge>
4. CPU - Screeps Wiki, accessed February 24, 2026, <https://wiki.screepspl.us/CPU/>
5. Screeps #15: Room Planning | Field Journal, accessed February 24, 2026, <https://www.jonwinsley.com/notes/screeps-room-planning>
6. Overmind - Ben Bartlett, accessed February 24, 2026, <https://bencbartlett.com/projects/overmind/>
7. Screeps #1: Overlord overload - Ben Bartlett, accessed February 24, 2026, <https://bencbartlett.com/blog/screeps-1-overlord-overload/>
8. Screeps #6: Verifiably refreshed | Ben Bartlett, accessed February 24, 2026, <https://bencbartlett.com/blog/screeps-6-verifiably-refreshed/>
9. So is everyone just using Overmind now?? : r/screeps - Reddit, accessed February 24, 2026, https://www.reddit.com/r/screeps/comments/cm5o0w/so_is_everyone_just_using_overmind_now/
10. Looking For Efficiency Tips : r/screeps - Reddit, accessed February 24, 2026, https://www.reddit.com/r/screeps/comments/ocdfg8/looking_for_efficiency_tips/
11. JonathanSafer/screeps: Screeps AI - GitHub, accessed February 24, 2026, <https://github.com/JonathanSafer/screeps>
12. Screeps #21: Patrolling the Perimeter - Field Journal, accessed February 24, 2026, <https://jonwinsley.com/notes/screeps-patrolling-perimeter>
13. bonzaiferroni/bonzAI-framework - GitHub, accessed February 24, 2026, <https://github.com/bonzaiferroni/bonzAI-framework>
14. bonzaiferroni/bonzAI - GitHub, accessed February 24, 2026, <https://github.com/bonzaiferroni/bonzAI>
15. On the topic of open source code bases | Screeps Forum, accessed February 24, 2026,

- <https://screeps.com/forum/topic/2000/on-the-topic-of-open-source-code-base-s>
16. ScreepsQuorum/screeps-quorum: Screeps Self Managing and Playing Codebase - GitHub, accessed February 24, 2026,
<https://github.com/ScreepsQuorum/screeps-quorum>
 17. Structure Layout · Issue #6 · ScreepsQuorum/screeps-quorum - GitHub, accessed February 24, 2026,
<https://github.com/ScreepsQuorum/screeps-quorum/issues/6>
 18. How does CPU usage calculation work? : r/screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/pmgrp8h/how_does_cpu_usage_calculation_work/
 19. Confusing information - Memory CPU use in parsing | Screeps Forum, accessed February 24, 2026,
<https://screeps.com/forum/topic/848/confusing-information-memory-cpu-use-in-parsing>
 20. CPU Usage with require | Screeps Forum, accessed February 24, 2026,
<https://screeps.com/forum/topic/743/cpu-usage-with-require>
 21. bonzaiferroni/Traveler: Traveler - A general movement solution for Screeps.com - GitHub, accessed February 24, 2026, <https://github.com/bonzaiferroni/Traveler>
 22. How to determine CPU usage | Screeps Forum, accessed February 24, 2026,
<https://screeps.com/forum/topic/3294/how-to-determine-cpu-usage>
 23. What does the game/CPU time mean for me? - screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/5l1nvz/what_does_the_gamecpu_time_mean_for_me/